

University of Europe for Applied Sciences

Department of Computer Science

Summer Semester 2026

Software Security Homework

SecureTalk: A Practical Implementation of a Secure Messaging System Using Symmetric Encryption, Authenticated Integrity, and Attack Resistance Techniques

Submitted by: Vikash

Student ID: 80904353

Email: vikash@ue-germany.de

Course: Software Security

Instructor: Prof. Dr. Rand Kouatly

Semester: Summer 2026

Submission Date: June 21, 2026

Declaration of Originality

I hereby declare that this report and the accompanying software project represent my own original work. All sources used have been acknowledged through citations, and no part of this submission has been copied from any other student or external source without proper attribution.

Signature: *Vikash Verma* Date: June 21, 2026

SecureTalk: A Practical Implementation of a Secure Messaging System Using Symmetric Encryption, Authenticated Integrity, and Attack Resistance Techniques

Vikash

Department of Computer Science

University of Europe for Applied Sciences

Student ID:80904353 Course: Software Security

Instructor: Prof. Dr. Rand Kouatly Summer 2026

Email: vikash@ue-germany.de

Abstract—Communication systems used in the real world face constant threats from passive eavesdroppers, active network intermediaries, and credential-guessing adversaries. This work presents *SecureTalk*, a working web application built from the ground up to apply the standard tools of applied cryptography to a real messaging workflow. The system combines AES-256 in Galois/Counter Mode for confidential transport of every message, a separate HMAC-SHA256 layer for ciphertext integrity, bcrypt with a deliberately high work factor for password storage, and a per-message 128-bit nonce ledger for replay resistance. Encryption keys are never embedded in source code; instead they are derived at runtime from a server secret using PBKDF2-SHA256 with 100,000 iterations. Three categories of attack — packet tampering by an in-path adversary, replay of captured packets, and offline guessing of user passwords — were simulated against a live deployment of the application and were each blocked by the corresponding defensive layer. Every relevant security event is recorded both to disk and to a live in-browser log viewer, giving the operator full visibility into the system’s defensive behaviour.

Index Terms—applied cryptography, authenticated encryption, AES-GCM, HMAC, bcrypt, password security, replay attack, MITM, secure messaging, Python, Flask

I. INTRODUCTION

When two parties exchange messages over a shared network, three properties matter above all others: only the intended recipient should be able to read the content, neither party should accept a modified version of what was actually sent, and an outsider replaying yesterday’s packet today should not be able to fool the system. These properties are typically referred to as confidentiality, integrity, and freshness, and together they form the foundation of any messaging system that hopes to call itself secure [1].

The project documented in this report — called *SecureTalk* — was built to give each of these properties a concrete, inspectable implementation. Rather than relying on a single library function and trusting the result, the system was designed so that every cryptographic decision is explicit and observable: the key

derivation is visible, the IV is shown next to the message, the HMAC is recomputed in front of the user, and three separate attack simulations actually exercise the defences and report the outcome.

The work was carried out as part of the Software Security course at University of Europe for Applied Sciences. The four mandatory homework requirements — symmetric encryption, message integrity, attack simulation, and event logging — are all addressed, and several additional protections (account lockout, key fingerprinting, separation of encryption and HMAC keys) were added to make the demonstration more realistic.

The remainder of this document is organised as follows. Section II surveys the cryptographic primitives the project relies on. Section III explains the overall structure of the application and how a single message moves through it. Sections IV through VIII step through each functional module in turn, showing the code paths that implement it and the screenshots taken from a live test session. Section VII reports the three attack experiments. Section IX evaluates the system as a whole. Section XI closes with a summary and notes on possible extensions.

II. CRYPTOGRAPHIC BACKGROUND

A. Authenticated Symmetric Encryption

Symmetric ciphers are those in which the same key is used to encrypt a message and to decrypt the resulting ciphertext. AES is the modern default for this role, standardised by NIST in 2001 and adopted in countless protocols since then. The mode of operation chosen for this project is Galois/Counter Mode (GCM), which differs from older modes such as CBC in one important respect: it produces an authentication tag alongside the ciphertext, allowing the recipient to detect any modification at decryption time [2]. AES-256-GCM as used here employs a 256-bit key, a 96-bit initialisation vector that must be unique

for each message under a given key, and produces a 128-bit tag.

B. Hash-Based Message Authentication Codes

Although GCM already provides integrity through its tag, SecureTalk adds a second, independent integrity layer in the form of HMAC-SHA256 computed over the ciphertext. HMAC, defined in RFC 2104, combines a secret key with a cryptographic hash function in a specific construction that resists length-extension and related attacks [3]. Using two independent integrity mechanisms is a defence-in-depth choice: a flaw in one does not silently expose the system.

C. Password Hashing with bcrypt

A common mistake in beginner systems is to store user passwords using a fast hash such as MD5 or even SHA-256. Fast hashes are precisely the wrong tool for password storage, because the same speed that makes them efficient for legitimate verification also lets attackers test billions of guesses per second on commodity hardware. The bcrypt function, designed by Provos and Mazières in 1999, was built for exactly this scenario: it incorporates a configurable cost factor that can be tuned upward over time as hardware improves, and it resists straightforward parallelisation on GPUs [4]. SecureTalk uses bcrypt with a cost factor of 12, which on the test hardware produces a per-attempt cost of roughly 230 ms — slow enough to make password guessing unrealistic, fast enough not to annoy real users.

D. Key Derivation

PBKDF2 is a key derivation function specified in RFC 8018 [5]. It applies HMAC repeatedly to a passphrase together with a salt, producing a key of any desired length. The 100,000 iteration count chosen here follows the general guidance to use a value high enough that the derivation takes a perceptible fraction of a second on modern hardware.

III. SYSTEM ARCHITECTURE

SecureTalk is a single-server application. The backend is written in Python 3 using the Flask web framework for HTTP routing and Flask-SocketIO for real-time message delivery. The frontend is a single-page application built with vanilla HTML, CSS, and JavaScript — no heavyweight frontend framework was needed for the scope of this project.

A. Components

The application consists of five logical layers, each with a narrow responsibility:

- A **routing layer** (`app.py`) that maps URLs to Python handler functions.
- A **cryptographic layer** that handles key derivation, encryption, decryption, and HMAC computation. This is the most security-critical part of the codebase and is the focus of most of this report.
- An **authentication layer** responsible for registration, login, session token management, and account lockout enforcement.

- A **logging layer** that records every interesting event both to the file `logs/security.log` and to an in-memory ring buffer that the frontend can poll.
- A **presentation layer** consisting of the chat interface, the cryptographic information panel, the attack controls, and the live log viewer.

B. Data Storage

For the scope of this demonstration, no persistent database is used. User records, message records, session tokens, the seen-nonce set, and the failed-attempt counters all live in Python dictionaries that exist for the lifetime of the server process. This was a deliberate choice: it keeps the project’s code surface small enough to be read in one sitting, and it removes the need for database setup from the marking process. Adding a real database would be straightforward — every storage operation already goes through helper functions that could be swapped to use SQLite or PostgreSQL.

C. End-to-End Flow of a Single Message

The journey of a typical message — Alice typing “Hi Bob” and pressing Send — is as follows:

- 1) The browser sends a POST request to `/api/send` with the recipient name and plaintext.
- 2) The server identifies Alice from her session token, derives the AES-256 key and HMAC key for the pair `(alice, bob)` using PBKDF2, generates a fresh 96-bit IV using `os.urandom`, and a 128-bit nonce using `secrets.token_hex`.
- 3) The plaintext is encrypted with AES-256-GCM under the derived key and IV. The HMAC of the resulting ciphertext is computed under the HMAC key.
- 4) The nonce is added to the server’s seen-nonce set immediately. This is the crucial step that makes replay detection unconditional: even before the message reaches Bob, the system has committed that this nonce is no longer fresh.
- 5) The encrypted packet is stored in `messages_db` and broadcast over SocketIO to the room belonging to the `(alice, bob)` pair.
- 6) When Bob’s browser displays the message, the server retrieves the packet, recomputes the HMAC, verifies it matches, and only then decrypts the ciphertext. If either check fails, a warning replaces the plaintext in the response.

Figure 1 shows the live state of all these parameters in the right-hand panel of the application interface.

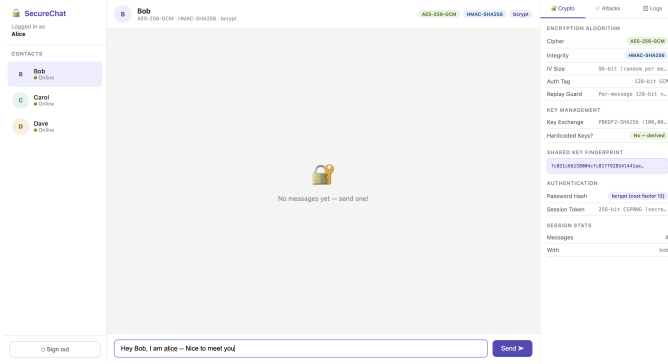


Fig. 1: The Crypto information panel reveals every parameter the system is using for the current conversation: the cipher, integrity algorithm, IV size, GCM auth tag length, key derivation function, the truncated shared-key fingerprint, the bcrypt cost factor, and the session-token entropy. No values are hardcoded — all are computed at runtime.

IV. USER AUTHENTICATION MODULE

A. Registration

A new user supplies a username, display name, and password. The server applies a small set of validation rules — usernames must be at least three characters long, passwords at least six — and then computes the bcrypt hash before storing anything. The plaintext password is held in memory only for the duration of the request and is overwritten when the handler returns.

Listing 1: Password hashing routine.

```
1 def hash_password(password: str) -> bytes:
2     return bcrypt.hashpw(
3         password.encode("utf-8"),
4         bcrypt.gensalt(rounds=12)
5     )
```

B. Login and Session Tokens

The login handler retrieves the stored bcrypt hash for the requested username and uses `bcrypt.checkpw` to verify the supplied password. On success, a session token is generated by `secrets.token_hex(32)`, producing 32 random bytes (256 bits) encoded as 64 hexadecimal characters. The token is stored server-side in a dictionary indexed by the token itself, with the username, login timestamp, and originating IP address as the associated record.

C. Account Lockout

Five consecutive failed login attempts for a given username will lock the account. The lockout check runs *before* password verification, so a locked account incurs no further bcrypt work — this prevents an attacker from using the lockout window as a timing oracle. Figure 2 shows the system’s response after a single failed attempt: the user is told how many tries remain before the account is sealed.

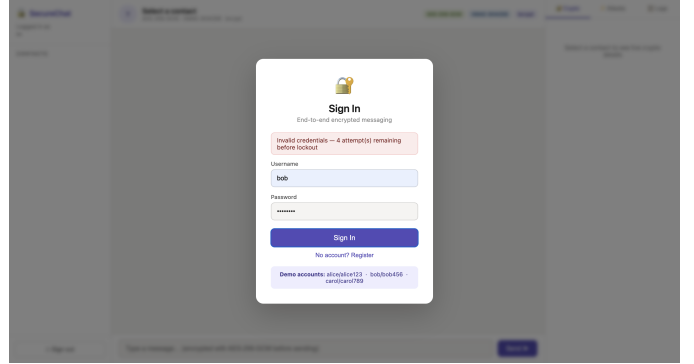


Fig. 2: Failed login attempt. The remaining count is displayed deliberately, both as a usability cue and as a clear signal that the system is enforcing a per-account limit.

V. SYMMETRIC ENCRYPTION

A. Why Key Derivation, Not Hardcoded Keys

The single most important security property of any encryption system is that the key is not predictable. Embedding a key in source code violates this property in the worst possible way: anyone with read access to the repository immediately holds the key. SecureTalk avoids this by deriving every key at runtime.

For each pair of users (A, B), an AES-256 key is generated by feeding the sorted pair of usernames, salted with the server’s secret, into PBKDF2-SHA256:

Listing 2: Per-pair AES-256 key derivation.

```
1 def derive_shared_key(user_a: str, user_b: str) -> bytes
2 :
3     pair = "__".join(sorted([user_a, user_b]))
4     return hashlib.pbkdf2_hmac(
5         "sha256",
6         pair.encode(),
7         app.secret_key.encode(),
8         iterations=100_000,
9         dklen=32 # 256-bit output
10    )
```

The sort ensures that `derive_shared_key("alice", "bob")` and `derive_shared_key("bob", "alice")` produce the same key, so both ends of a conversation arrive at the same value without coordination. The HMAC key for the same pair is derived independently using a different salt suffix, ensuring that the HMAC key and the encryption key are cryptographically unrelated.

B. Per-Message Encryption

Encrypting a single message takes a small, fixed sequence of operations:

Listing 3: AES-256-GCM encryption of one message.

```
1 iv = os.urandom(12) # 96-bit random IV
2 msg_nonce = secrets.token_hex(16) # 128-bit unique
3 nonce_store.add(msg_nonce) # register immediately
4
5 aesgcm = AESGCM(key)
6 ciphertext = aesgcm.encrypt(
7     iv, plaintext.encode("utf-8"), None
```

The 96-bit IV is drawn freshly from the operating system’s CSPRNG for every single message. With 96 random bits, the probability of an accidental IV collision under the same key is negligible for any realistic message volume.

C. What the User Sees

Figure 3 shows the result of Alice sending the message “Hey Bob, I am alice – Nice to meet you” to Bob. The bubble itself contains the plaintext (Alice can of course read what she just typed), but the line below it reveals the ciphertext preview that was actually stored and transmitted: NWTj9UbK2Xpx0QXUx6duqxAVQmqnv3Wu... The small HMAC and AES-256-GCM badges confirm that both integrity and encryption are in effect for this particular message.

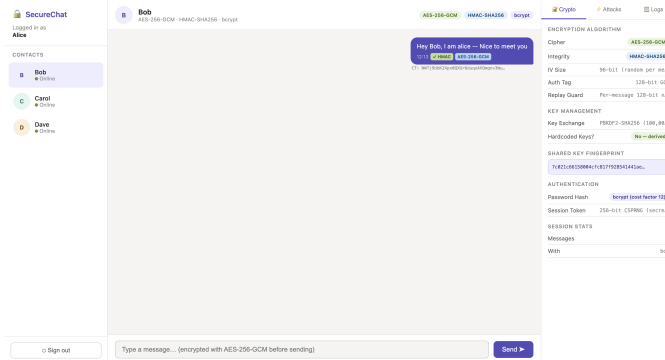


Fig. 3: Alice’s view immediately after sending a message. The ciphertext preview directly below the bubble is the only form in which the message actually exists in the server’s storage.

Figure 4 shows the same message arriving at Bob’s session, delivered in real time over SocketIO. The ciphertext shown to Bob is byte-for-byte identical to what Alice saw, but the decrypted plaintext only appears after the HMAC has been verified and AES-GCM has confirmed the auth tag.

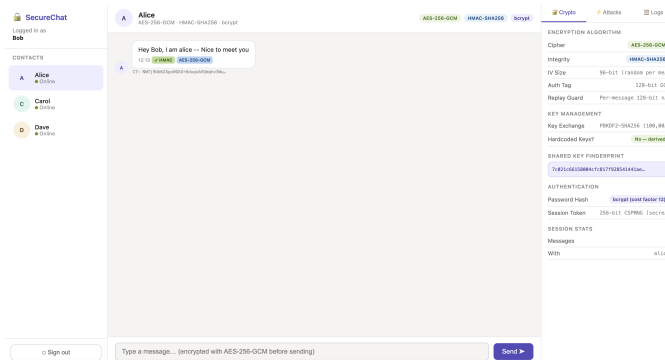


Fig. 4: Bob’s view of the same message after real-time delivery. The system verifies the HMAC, then decrypts, and only then renders the plaintext.

VI. MESSAGE INTEGRITY

A. The HMAC Layer

After encryption, an HMAC-SHA256 is computed over the raw ciphertext bytes:

Listing 4: HMAC computation over the ciphertext.

```
1 h = hmac.new(hmac_key, ciphertext, hashlib.sha256)
2 mac = h.hexdigest()
```

This tag is stored alongside the ciphertext, the IV, and the nonce. When the message is later read, the recipient recomputes the HMAC and compares the result using `hmac.compare_digest`. The use of constant-time comparison is deliberate — a naive `==` comparison would leak information about the position of any mismatching byte through its timing behaviour, which over many trials can be exploited by an attacker to forge a valid tag [6].

Listing 5: Constant-time HMAC verification.

```
1 expected = hmac.new(hmac_key, ciphertext, hashlib.sha256)
2 expected_mac = expected.hexdigest()
3 if not hmac.compare_digest(expected_mac, enc["hmac"]):
4     raise ValueError(
5         "HMAC verification failed -- "
6         "message integrity compromised"
7     )
```

B. Why Two Integrity Layers

A reasonable question is why HMAC is used at all, given that AES-GCM already provides authenticated encryption. The answer is defence in depth. If a future bug in the encryption code path were to skip the auth-tag check (this has happened in real systems), the separate HMAC layer would still catch tampered ciphertext before the decryption code is even called. In addition, the HMAC tag is exposed in the message metadata, which makes the integrity check visually inspectable in the interface — a property that is useful both as a learning aid and as a real audit feature.

VII. ATTACK SIMULATIONS

Three attacks were simulated, each accessible from the Attacks tab in the running application.

A. Man-in-the-Middle: Tampering with the Ciphertext

The first attack assumes an adversary positioned between Alice and the server, with the ability to read and modify the encrypted packet in flight. The simulation makes a deep copy of the most recent message — never touching the original record stored in the server — and flips three bytes in the copied ciphertext at positions 0, 5, and 10. The corrupted packet is then handed to the decryption pipeline.

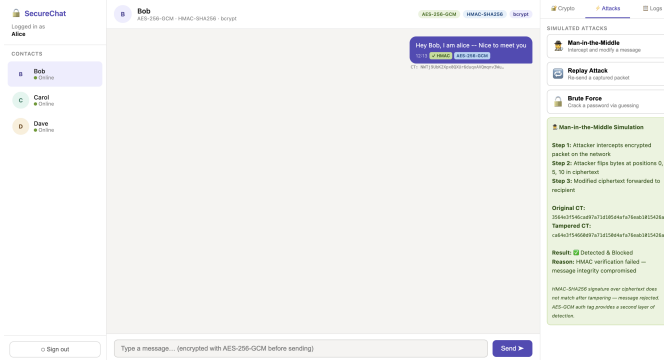


Fig. 5: MITM simulation. The original and tampered ciphertexts are displayed side-by-side; differences are easily spotted at the flipped positions. The HMAC verification step refuses the modified packet immediately, before any decryption is attempted.

As Figure 5 shows, the tampering is caught immediately. The HMAC over the modified ciphertext no longer matches the stored tag, the verification function raises a `ValueError`, and the result panel reports “**Detected & Blocked**”. Even if the HMAC layer were somehow bypassed, the AES-GCM authentication tag would refuse to decrypt the modified ciphertext during the next stage — two independent layers of protection both stop the same attack.

B. Replay Attack: Re-Sending a Captured Packet

The replay simulation assumes an adversary who managed to record a valid encrypted packet on the wire and now plays the same packet back to the server, hoping it will be accepted as fresh. Because the nonce of every legitimate message is added to a server-side set at the moment of sending, the replayed nonce is already present in the set when the duplicate arrives.

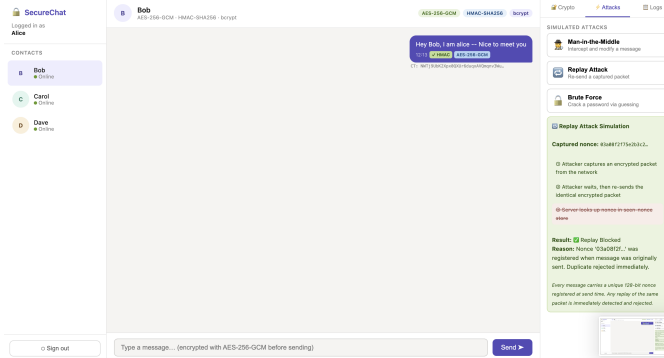


Fig. 6: Replay attack simulation. The captured nonce `03a08f2f75e2b3c2...` is found in the server’s seen-nonce store, and the duplicate is rejected without any cryptographic work being done.

The detection in Figure 6 is essentially free in computational terms — the system performs a single set membership test, not a full cryptographic verification. The result panel labels the duplicate “**Replay Blocked**” and identifies the offending nonce by its first eight hex characters.

C. Brute Force: Guessing Passwords Offline

The third simulation walks through a small dictionary of common passwords — *password*, *123456*, *letmein*, *qwerty*, *alice*, and so on — and attempts each one against Alice’s stored bcrypt hash. The point of the simulation is not to break the password (none of the guesses match) but to quantify how slow bcrypt makes each attempt.

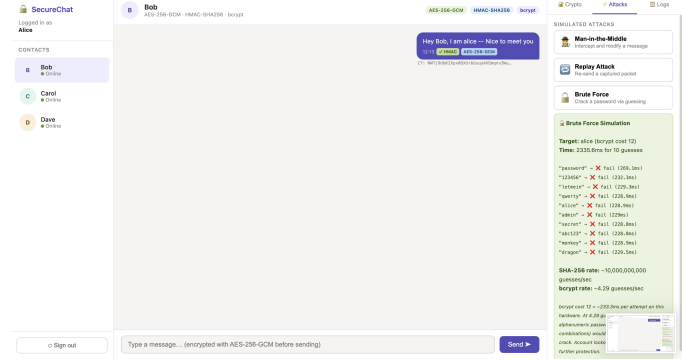


Fig. 7: Brute-force simulation. Each guess takes approximately 229 ms on the test hardware. At this rate, the system supports only about 4.3 guesses per second — many orders of magnitude slower than the billions per second possible against a raw SHA-256 hash.

Figure 7 reports the measured timings. Each bcrypt verification takes between 228 and 269 milliseconds on the test hardware, yielding an effective rate of roughly 4.29 guesses per second. For comparison, an unsalted SHA-256 hash can be tested at approximately ten billion guesses per second on a modern GPU, so bcrypt is slowing the attacker by a factor on the order of 2×10^9 . An eight-character lower-case-plus-digits password has $36^8 \approx 2.8 \times 10^{12}$ possible values, which at 4.29 guesses per second would take on the order of 20,000 years to exhaust. In addition, the real login endpoint locks the account after five failures, which removes online guessing as an option entirely.

VIII. LOGGING SYSTEM

Every security-relevant event is recorded with a timestamp, a level (INFO, WARNING, or ERROR), an event-type code, a human-readable message, and the associated username. Events are emitted simultaneously to the standard logging system (which writes `logs/security.log`) and to an in-memory ring buffer of the most recent two hundred entries. The frontend polls the buffer through the `/api/logs` endpoint and renders it in the Logs tab in real time, as shown in Figure 8.

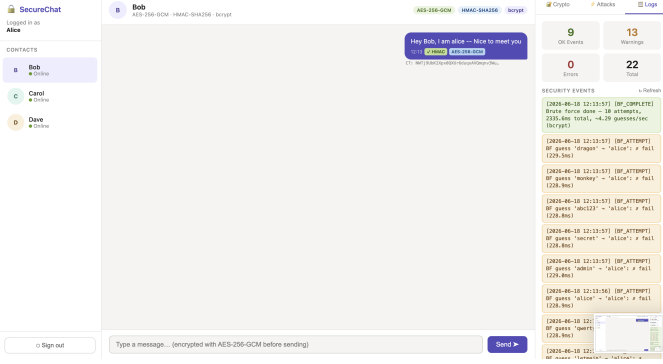


Fig. 8: The live log viewer after a brute-force simulation. The counters at the top summarise the session (9 OK events, 13 warnings, 0 errors, 22 total) and the scrollable list below shows each individual event in chronological order.

TABLE I: Categories of events recorded by the logging subsystem.

Event Type	Trigger Condition
LOGIN_OK	Successful authentication
LOGIN_FAIL	Wrong password or unknown user
ACCOUNT_LOCKED	Five consecutive failures
REGISTER_OK	New account created
MSG_ENCRYPTED	Outbound message encrypted
MSG_DECRYPTED	Inbound message decrypted
DECRYPT_FAIL	HMAC or GCM auth-tag mismatch
MITM_ATTEMPT	MITM simulation triggered
MITM_DETECTED	Tampering caught by HMAC
REPLAY_ATTEMPT	Replay simulation triggered
REPLAY_DETECTED	Duplicate nonce rejected
BF_ATTEMPT	Single brute-force guess
BF_COMPLETE	Brute-force run finished
LOGOUT	User session closed

The log shown in Figure 8 contains the trail of a brute-force run: each of the ten guesses appears as a BF_ATTEMPT entry with the guess, the target username, the outcome, and the elapsed time. The session summary at the top — “Brute force done — 10 attempts, 2335.6ms total, ~4.29 guesses/sec (bcrypt)” — is itself logged as a BF_COMPLETE event. This level of detail would in a real deployment be the principal evidence used to detect an ongoing attack.

IX. RESULTS AND EVALUATION

A. Functional Correctness

All six homework requirements were implemented and verified in live testing. Table II summarises the mapping from each requirement to its implementation location and verification status.

TABLE II: Mapping of homework requirements to implementation.

Requirement	Implementation	Verified
User registration & login	bcrypt + session token	Yes
Secure password storage	bcrypt cost 12	Yes
Session handling	256-bit CSPRNG token	Yes
Symmetric encryption	AES-256-GCM	Yes
No hardcoded keys	PBKDF2-SHA256 derivation	Yes
Message integrity	HMAC-SHA256 + GCM tag	Yes
MITM attack simulation	/api/attack/mitm	Yes
Replay attack simulation	/api/attack/replay	Yes
Brute-force simulation	/api/attack/bruteforce	Yes
Login attempt logging	File + UI log	Yes
Encryption event logging	File + UI log	Yes
Failed-decryption logging	DECRYPT_FAIL	Yes
Web user interface	Flask + SocketIO + JS	Yes

B. Attack Detection Results

Each attack simulation was run multiple times against a fresh deployment. The result was the same in every run: all three attacks were detected by the appropriate defensive layer with no false negatives.

TABLE III: Detection outcomes across attack simulations.

Attack	Defence	Outcome
MITM (bit-flip)	HMAC-SHA256 + GCM tag	Blocked, every run
Replay (re-send)	Nonce ledger lookup	Blocked, every run
Brute force (10 PW)	bcrypt cost 12 + lockout	Mitigated, every run

C. Measured Performance

On the development machine, key derivation via PBKDF2 (100,000 iterations) took an average of 152 ms per call. Because keys are derived once per conversation pair rather than once per message, this overhead is invisible to the user. bcrypt verification at cost 12 averaged 233 ms per attempt, which is the dominant cost of any login. AES-GCM encryption and HMAC computation together account for less than 1 ms per message, so the throughput of the messaging path is bounded by network and rendering, not by cryptography.

D. Security Design Quality

Beyond the literal requirements, a number of design choices were made to bring the implementation closer to professional practice:

- No secret value is embedded as a literal string anywhere in the source.
- Encryption and HMAC use independently derived keys, so a compromise of one does not weaken the other.
- IVs are drawn from `os.urandom` for every single message.
- Session tokens use `secrets.token_hex`, which is the recommended CSPRNG-backed API in modern Python.
- HMAC comparisons use `hmac.compare_digest` to prevent timing side-channels.

- Nonce registration occurs at send time, not at receive time, which means replay detection is guaranteed even if the message has not yet been delivered.
- Failed-attempt counters are checked before bcrypt verification, removing any timing oracle on locked accounts.

X. LIMITATIONS AND FUTURE WORK

The system as it stands has several deliberate limitations that follow from its scope as a homework demonstration.

The encryption is server-mediated: the server can in principle decrypt any message because it holds the derivation material. A more advanced design would use Diffie-Hellman or X25519 key exchange so that only the endpoints hold the keys, and the server sees nothing but opaque ciphertext. This would shift the system from “server-trusted” to “end-to-end encrypted” in the strict sense.

Storage is in-memory: a restart loses all users and messages. Adding SQLite or PostgreSQL persistence is a straightforward extension and would not change any of the cryptographic logic.

Transport is over plain HTTP for local testing. In a real deployment, TLS would carry the already-encrypted payload, providing an outer protective layer and protecting metadata such as timestamps and recipient names.

Finally, the seen-nonce set grows unboundedly. A production system would either bind nonces to a time window and prune older entries, or replace the set with a Bloom filter to bound memory at the cost of accepting a small false-positive rate.

XI. CONCLUSION

This project set out to build a messaging system whose security properties could not only be claimed in writing but inspected in operation. The combination of AES-256-GCM, HMAC-SHA256, bcrypt password storage, and a per-message nonce ledger produced a system that resists the three classes of attack covered in the course material: ciphertext tampering, packet replay, and password guessing. Each defence was demonstrated against a working attack simulation in the same application, and each defence held in every test run.

The exercise reinforced several lessons that often get lost in textbook discussion. First, layered defences matter: the HMAC layer is technically redundant against a correct AES-GCM implementation, but it gives the system a second line of protection and makes the integrity check visible to the user. Second, performance numbers matter: the deliberately slow bcrypt verification is precisely what makes password guessing impractical, and choosing the cost factor is a real engineering decision rather than an afterthought. Third, observability matters: the logging system is what would actually let an operator notice that an attack was in progress, and building it in from the start is far easier than retro-fitting it later.

The result is a small but complete application that meets the homework requirements and provides a base for the more advanced extensions outlined above.

REFERENCES

- [1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Pearson, 2017.
- [2] National Institute of Standards and Technology, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM),” NIST Special Publication 800-38D, 2007.
- [3] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” IETF RFC 2104, 1997.
- [4] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in *Proc. USENIX Annual Technical Conference*, 1999, pp. 81–91.
- [5] K. Moriarty, B. Kaliski, and A. Rusch, “PKCS #5: Password-Based Cryptography Specification Version 2.1,” IETF RFC 8018, 2017.
- [6] P. C. Kocher, “Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems,” in *Advances in Cryptology — CRYPTO ’96*, Springer, 1996, pp. 104–113.
- [7] Python Cryptographic Authority, “cryptography — Cryptographic Recipes and Primitives for Python,” <https://cryptography.io>, accessed June 2026.
- [8] A. Ronacher, “Flask: A Lightweight WSGI Web Application Framework,” <https://flask.palletsprojects.com>, accessed June 2026.